МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА ВТ

ОТЧЁТ

по лабораторной работе №1

Тема: Методы неинформированного (слепого) поиска

Студент гр. 3312	_____	Барченков П.А.
Студент гр. 3312	_____	Лебедев И.А.
Студент гр. 3312	_____	Шарапов И.Д.
Преподаватель	_____	Пантелеев М.Г.

Санкт-Петербург

2026

Оглавление

Цель работы 3

Постановка задачи (Вариант 1) 3

Описание выбранных структур данных, представление функции определения последователей 3

Описание алгоритма 5

Исходный код 6

Результат работы 9

Вывод 13

Цель работы

Практическое закрепление понимания общих идей поиска в пространстве состояний и стратегий слепого поиска.

Постановка задачи (Вариант 1)

В работе рассматривается задача поиска решения головоломки 8-ки. Игровое поле представляет собой квадрат 3×3, на котором расположены восемь пронумерованных фишек и одна пустая клетка, обозначаемая числом 0. За один ход разрешается переместить в пустую клетку одну из соседних по вертикали или горизонтали фишек.

Для варианта 1 требуется найти последовательность ходов, переводящую начальное состояние 583402761 в целевое состояние 123456780. Состояния в программе представлены в виде девятизначного числа, где каждая цифра соответствует содержимому одной позиции поля.

Для решения задачи необходимо реализовать методы неинформированного поиска – поиск в ширину (BFS) и поиск в глубину (DFS), обеспечить построение последовательности состояний от начального к целевому и получить экспериментальные оценки временной и ёмкостной сложности работы алгоритмов.

Описание выбранных структур данных, представление функции определения последователей

В программе состояние головоломки представляется в виде девятизначного целого числа. Каждая цифра соответствует содержимому одной позиции поля 3×3. Начальное состояние задаётся константой START_STATE = 583402761, а целевое – GOAL_STATE = 123456780. Такой способ позволяет компактно хранить состояние и выполнять преобразования без использования массивов.

Для обращения к отдельным элементам состояния используется функция extract_digit(x, i), которая извлекает цифру, находящуюся в позиции i.

Формирование нового состояния после перемещения фишки выполняется функцией swap_in_int(x, i, j), которая меняет местами пустую клетку и соседнюю с ней фишку.

Допустимые переходы между клетками задаются словарём pos_from_pos. Для каждой позиции в нём указан список соседних позиций, в которые может быть перемещена пустая клетка. Например, для центральной клетки с индексом 4 допустимы переходы в позиции 1, 3, 5 и 7.

Функция определения последователей реализована как gen_children(x). Она находит положение пустой клетки, после чего по словарю pos_from_pos строит список всех состояний, которые могут быть получены за один допустимый ход.

Для хранения вершин, ожидающих раскрытия, используется двусторонняя очередь deque из модуля collections. Она выбрана потому, что позволяет эффективно добавлять и извлекать элементы как из начала, так и из конца. В режиме поиска в ширину новые вершины добавляются в конец очереди методом append, а в режиме поиска в глубину – в начало методом appendleft. Извлечение текущей вершины в обоих случаях выполняется методом popleft. Это позволяет реализовать BFS и DFS на основе одной и той же структуры данных.

Для хранения информации о построенном пути используется словарь history. В нём для каждой посещённой вершины сохраняется её родитель. Это позволяет после нахождения целевого состояния восстановить полный путь от цели к начальному состоянию. Кроме того, наличие вершины в словаре history используется как признак того, что данное состояние уже было посещено, что предотвращает повторное добавление одинаковых вершин в очередь.

Дополнительно в программе реализована проверка разрешимости задачи с помощью функций count_inversions(x) и is_solvable(start, goal). Проверка основана на подсчёте числа инверсий и позволяет до запуска поиска определить, существует ли решение для заданных начального и целевого состояний.

Описание алгоритма

В программе реализованы два метода неинформированного поиска: поиск в ширину (BFS) и поиск в глубину (DFS). Оба алгоритма работают в пространстве состояний головоломки 8-ки, где вершинами являются конфигурации игрового поля, а рёбрами – допустимые перемещения пустой клетки.

В начале работы в очередь помещается начальное состояние. Для него в словаре history сохраняется ссылка на самого себя, что позволяет в дальнейшем восстановить путь к решению. Затем выполняется цикл поиска: из очереди извлекается текущая вершина, проверяется её совпадение с целевым состоянием, после чего для неё строятся все допустимые потомки с помощью функции gen_children(x).

Если текущая вершина совпадает с целевым состоянием, поиск завершается. После этого программа восстанавливает путь от цели к начальному состоянию, последовательно переходя по словарю history от каждой вершины к её родителю. Полученная последовательность затем выводится пользователю.

Если текущая вершина не является целевой, программа перебирает все её дочерние состояния. Для каждого потомка проверяется, встречался ли он ранее. Если состояние ещё не посещалось, оно заносится в словарь history, а затем добавляется в очередь на дальнейшее рассмотрение. Если состояние уже присутствует в history, повторно в очередь оно не добавляется. Это позволяет избежать циклов и повторного раскрытия одинаковых вершин.

Различие между BFS и DFS в программе определяется способом добавления новых вершин в очередь. В алгоритме поиска в ширину новая вершина помещается в конец очереди методом append, поэтому состояния раскрываются в порядке возрастания глубины. В алгоритме поиска в глубину новая вершина помещается в начало очереди методом appendleft, поэтому поиск в первую очередь углубляется по последнему найденному направлению. Извлечение текущей вершины в обоих случаях выполняется методом popleft.

Таким образом, алгоритм BFS обеспечивает последовательный просмотр дерева поиска по уровням и позволяет находить решение минимальной глубины, тогда как алгоритм DFS ориентирован на быстрое углубление в дерево, но не гарантирует кратчайшего решения. В программе также ведётся подсчёт числа шагов поиска и количества посещённых состояний, что позволяет получить экспериментальные оценки временной и ёмкостной сложности.

Исходный код

```
from collections import deque
import os

pos_from_pos = {0: [1, 3],
                1: [0, 2, 4],
                2: [1, 5],
                3: [0, 4, 6],
                4: [1, 3, 5, 7],
                5: [2, 4, 8],
                6: [3, 7],
                7: [4, 6, 8],
                8: [5, 7]
                }

START_STATE = 583402761
GOAL_STATE = 123456780

def clear_console():
    os.system('cls' if os.name == 'nt' else 'clear')


def extract_digit(x: int, i: int) -> int:
    return x % 10 ** (9 - i) // 10 ** (8 - i)


def swap_in_int(x: int, i: int, j: int) -> int:
    return x + extract_digit(x, j) * 10 ** (8 - i) - extract_digit(x, j) * 10 ** (8 -
j)


def gen_children(x: int):
    for i in range(9):
        if extract_digit(x, i) == 0:
            ans = list()
            for j in pos_from_pos[i]:
                ans.append(swap_in_int(x, i, j))
            return ans
    return None


def count_inversions(x: int) -> int:
    arr = list()
    for i in range(9):
        d = extract_digit(x, i)
```

```
        if d != 0:
            arr.append(d)

    inv = 0
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] > arr[j]:
                inv += 1
    return inv


def is_solvable(start: int, goal: int) -> bool:
    return count_inversions(start) % 2 == count_inversions(goal) % 2


def grid_print(ans: list, step=-1):
    for i in range(3):
        for k in range(len(ans)):
            for j in range(3):
                print(extract_digit(ans[k], i * 3 + j), end=" ")
            print("  ", end="")
        if step != -1 and i == 1:
            print(f"Шаг {step}")
        else:
            print()


def trek_print(ans: list):
    step = 0
    for u in ans[:0:-1]:
        grid_print([u], step)
        step += 1
        print("  |\n  V")
    grid_print([ans[0]], step)


def solve(x: int, y: int, alg="BFS", mode="auto"):
    history = dict()
    q = deque()
    ans = list()
    q.append(x)
    history[x] = x
    time_compl = 0

    while len(q) > 0:
        u = q.popleft()
        time_compl += 1
        if u == y:
            while u != x:
                ans.append(u)
                u = history[u]
            ans.append(u)
            break
        children = gen_children(u)
        if mode == "manual":
            grid_print([u])
            print("  |")
            print("  " + "-" * (len(children) * 8 - 7))
            print("  |     " * len(children))
            grid_print(children)
        for v in children:
```

```
            if v not in history.keys():
                history[v] = u
                if alg == "BFS":
                    q.append(v)
                else:
                    q.appendleft(v)
                if mode == "manual":
                    print("     ", end="   ")
            elif mode == "manual":
                print("  ✓  ", end="   ")
        if mode == "manual":
            print()
            if input() == "0":
                return None
    return ans, len(history.keys()), time_compl


while True:
    clear_console()
    print('Начальное состояние:')
    grid_print([START_STATE])
    print('Целевое состояние:')
    grid_print([GOAL_STATE])
    print("Выберите алгоритм решения:\n"
          "1. BFS\n"
          "2. DFS\n\n"
          "0. Выйти")
    x = input()
    while x != "0" and x != "1" and x != "2":
        clear_console()
        print("Некорректный ввод!")
        print("Выберите алгоритм решения:\n"
              "1. BFS\n"
              "2. DFS\n\n"
              "0. Выйти")
        x = input()
    if x == "1":
        alg = "BFS"
    elif x == "2":
        alg = "DFS"
    else:
        break

    clear_console()
    print(f"Алгоритм \"{alg}\".\n"
          "Выберите режим работы:\n"
          "1. Пошаговый\n"
          "2. Автоматический\n\n"
          "0. Выйти")
    x = input()
    while x != "0" and x != "1" and x != "2":
        clear_console()
        print("Некорректный ввод!")
        print(f"Алгоритм \"{alg}\".\n"
              "Выберите режим работы:\n"
              "1. Пошаговый\n"
              "2. Автоматический\n\n"
              "0. Выйти")
        x = input()
    if x == "1":
        mode = "manual"
```

```
        clear_console()
        print(f"Алгоритм \"{alg}\", режим \"Пошаговый\".\n"
              "Для следующего шага нажмите любую клавишу, кроме \"0\"\n"
              "Чтобы вернуться в меню, нажмите \"0\".")
    elif x == "2":
        mode = "auto"
        clear_console()
        print(f"Алгоритм \"{alg}\", режим \"Автоматический\".")
    else:
        break

    if not is_solvable(START_STATE, GOAL_STATE):
        print("Данное начальное состояние неразрешимо для выбранного целевого
состояния.")
        print("Чтобы вернуться в меню, нажмите любую клавишу.")
        input()
        continue

    if mode == "manual":
        solve(START_STATE, GOAL_STATE, alg, mode)
    else:
        ans, space_compl, time_compl = solve(START_STATE, GOAL_STATE, alg, mode)
        trek_print(ans)
        print("----------------------------------------------")
        print(f"Ёмкостная сложность: {space_compl}")
        print(f"Временная сложность: {time_compl}")
        print(f"Стоимость пути: {len(ans) - 1}")
        print("----------------------------------------------")
        print("Чтобы вернуться в меню, нажмите любую клавишу.")
        input()
    continue
```

Результат работы

В результате выполнения программы были получены решения головоломки 8-ки с использованием алгоритмов поиска в ширину (BFS) и поиска в глубину (DFS). Программа позволяет выбрать алгоритм работы и режим выполнения: пошаговый или автоматический. В пошаговом режиме отображается процесс раскрытия вершин и построения последующих состояний, а в автоматическом режиме выводится найденный путь от начального состояния к целевому, а также экспериментальные оценки ёмкостной и временной сложности и стоимость найденного пути.

На рисунках ниже представлены основные этапы работы программы: выбор алгоритма, выбор режима, пример пошагового выполнения, а также результаты автоматического режима для BFS и DFS.

```
Выберите алгоритм решения:
1. BFS
2. DFS

0. Выйти
```

Рисунок 1 – Выбор алгоритма решения задачи

```
Алгоритм "BFS".
Выберите режим работы:
1. Пошаговый
2. Автоматический

0. Выйти
```

Рисунок 2 – Выбор режима работы программы

```
Алгоритм "BFS", режим "Пошаговый".
Для следующего шага нажмите любую клавишу, кроме "0"
Чтобы вернуться в меню, нажмите "0".
5 8 3
4 0 2
7 6 1
  |
  -------------------------
  |       |       |       |
5 0 3   5 8 3   5 8 3   5 8 3
4 8 2   0 4 2   4 2 0   4 6 2
7 6 1   7 6 1   7 6 1   7 0 1


5 0 3
4 8 2
7 6 1
  |
  -----------------
  |       |       |
0 5 3   5 3 0   5 8 3
4 8 2   4 8 2   4 0 2
7 6 1   7 6 1   7 6 1
                  ✓

5 8 3
0 4 2
7 6 1
  |
  -----------------
  |       |       |
0 8 3   5 8 3   5 8 3
5 4 2   4 0 2   7 4 2
7 6 1   7 6 1   0 6 1
         ✓
```

Рисунок 3 – Пример работы программы в пошаговом режиме для BFS

```
  |
  V
1 2 3
4 5 6   Шаг 20
0 7 8
  |
  V
1 2 3
4 5 6   Шаг 21
7 0 8
  |
  V
1 2 3
4 5 6   Шаг 22
7 8 0
----------------------------------------------
Ёмкостная сложность: 114058
Временная сложность: 90532
Стоимость пути: 22
----------------------------------------------
Чтобы вернуться в меню, нажмите любую клавишу.
```

Рисунок 4 – Результат работы программы в автоматическом режиме для BFS

```
  |
  V
1 2 3
4 0 6   Шаг 48434
7 5 8
  |
  V
1 2 3
4 5 6   Шаг 48435
7 0 8
  |
  V
1 2 3
4 5 6   Шаг 48436
7 8 0
----------------------------------------------
Ёмкостная сложность: 97731
Временная сложность: 55404
Стоимость пути: 48436
----------------------------------------------
Чтобы вернуться в меню, нажмите любую клавишу.
```

Рисунок 5 – Результат работы программы в автоматическом режиме для DFS

Вывод

В ходе выполнения лабораторной работы были изучены и реализованы два метода неинформированного поиска: поиск в ширину (BFS) и поиск в глубину (DFS) для решения головоломки 8-ки. Практические результаты показали, что оба алгоритма позволяют найти путь от начального состояния к целевому, однако их поведение и эффективность существенно различаются.

Алгоритм BFS показал себя лучше с точки зрения качества найденного решения. Он нашёл путь стоимостью 22 шага, тогда как алгоритм DFS нашёл путь стоимостью 48436 шагов. Это объясняется тем, что поиск в ширину раскрывает вершины по уровням и поэтому находит решение минимальной глубины, то есть кратчайший путь в пространстве состояний.

Алгоритм DFS, напротив, углубляется по одному из направлений поиска, не сравнивая его с другими возможными вариантами. Поэтому он может достаточно быстро дойти до некоторого решения, но это решение может оказаться очень длинным и неэффективным. В полученных результатах это проявилось особенно наглядно: несмотря на то, что DFS потребовал меньше шагов поиска (55404 против 90532 у BFS) и немного меньший объём памяти (97731 против 114058), найденный путь оказался во много раз хуже по стоимости. Таким образом, выигрыш DFS по экспериментальным оценкам времени и памяти в данном случае достигается ценой резкого ухудшения качества решения.

Следовательно, для 8-ки алгоритм BFS является более предпочтительным, если требуется получить оптимальный путь. Алгоритм DFS может быть полезен как более простой способ быстрого углубления в дерево поиска, однако для данной задачи он уступает BFS по качеству найденного результата. Практический эксперимент подтвердил теоретические свойства этих алгоритмов: BFS обеспечивает поиск кратчайшего решения, а DFS не гарантирует оптимальности и сильно зависит от порядка обхода вершин.

Таблица 1

Алгоритм	BFS	DFS
Стоимость пути	22	48436
Временная сложность	90532	55404
Ёмкостная сложность	114058	97731

По данным таблицы 1 видно, что BFS требует больше ресурсов, но обеспечивает значительно лучшее решение, тогда как DFS экономичнее по числу шагов поиска и по памяти, но приводит к крайне невыгодному пути. Поэтому итоговый выбор алгоритма зависит от цели: если важна оптимальность решения, лучше использовать BFS; если приоритетом являются меньшие затраты на поиск, можно рассматривать DFS, но с пониманием его ограничений.